ENGRAM

Persistent Memory Layer for Claude Code

VS Code Extension + Web Dashboard

Author	Abhi Khade
Product	Engram v1.0
Version	1.0 — Initial PRD
Date	April 2026
Status	Active Build
Hackathon	Built with Opus 4.7 — Apr 21–28, 2026
Format	VS Code Extension + Web Dashboard
Stack	TypeScript + ChromaDB + Claude API + React

CONFIDENTIAL — Hackathon Submission Document

1. Executive Summary

Engram is a VS Code extension that gives Claude Code a persistent, long-term memory of your codebase. Every time a developer opens a new Claude Code session, they waste 10–20 minutes re-explaining their project — the stack, the architecture, recent decisions, known bugs. Engram eliminates this entirely.

The extension silently indexes your codebase on first activation, records high-signal decisions from every Claude Code session, and automatically injects the right context at the start of every new session. Claude starts informed. No re-explaining. No lost context. No repeated bugs.

> *Engram does not wrap Claude Code. It fixes the fundamental infrastructure gap in how Claude Code operates today — the absence of memory across sessions.*

1.1 The One-Line Pitch

Claude Code is amnesiac. Engram is the brain it was never given.

1.2 Hackathon Context

Field	Detail
Hackathon	Built with Opus 4.7 — Global Virtual Hackathon
Dates	April 21 (12:00 PM EDT) – April 28 (8:00 PM EDT), 2026
Prize Pool	$100,000 in Claude API credits
Builder	Abhi Khade — Solo Developer
Category	Developer Tooling / Claude Code Infrastructure
Demo Project	DentaDesk (React Native dental clinic SaaS — 52 screens)

2. Problem Statement

Claude Code is the most capable agentic coding tool available today. But it has one fundamental flaw that affects every developer, every single session.

> *Claude Code has no memory. Every session is a blank slate. Every developer starts from zero every time.*

2.1 The Core Pain

A typical Claude Code session on an ongoing project begins like this:

```
Developer: "I'm working on DentaDesk, a React Native dental clinic app
using Expo SDK 52 with WatermelonDB for offline storage and Supabase for
cloud sync. I have 52 screens, and the billing module uses Razorpay. The
patient model has..."

15 minutes of context-setting later...

Developer: "...okay so can you help me add appointment reminders?"
```

This happens every single session. On every project. For every Claude Code user on the planet.

2.2 Problem Breakdown

Problem	Impact
Constant Re-explanation	Developers re-explain the same stack, architecture, and conventions every session — even on projects they've worked on for months.
Lost Decision Context	Why was this pattern chosen? Why does this file exist? The reasoning behind every architectural decision vanishes after each session ends.
Bugs Fixed Twice	The same class of bug gets reintroduced because Claude doesn't remember fixing it last week or why a particular approach was avoided.
Zero Personalization	Claude gives generic advice because it doesn't know your coding style, your constraints, or your past choices in this specific project.
Wasted Developer Hours	Across millions of Claude Code users globally, this represents an enormous amount of lost productivity — spent on setup instead of building.

3. Solution — How Engram Works

Engram introduces a three-layer memory architecture that runs silently alongside Claude Code. It is fully automatic — developers do nothing differently. Engram works in the background.

3.1 The Three Memory Layers

01

Codebase Indexer
On first activation, Engram deeply scans your entire project. It uses Tree-sitter to parse every file — extracting components, functions, routes, DB schema, dependencies, and architectural patterns. All extracted data is converted into semantic vector embeddings and stored in a local ChromaDB instance on your machine. Nothing leaves your computer.

02

Session Memory Recorder
As you work with Claude Code, Engram listens. It watches file diffs after each session to detect what changed and infers why. It extracts high-signal memories — bugs fixed, decisions made, patterns established, approaches avoided — and stores them as structured entries tagged by type, file, and timestamp.

03

Context Injector
When a new Claude Code session opens, Engram performs a semantic similarity search across all stored memories. It retrieves the most relevant entries for your current task, compresses them into a compact context block, and injects it into Claude's system prompt automatically. Claude starts the session already knowing your project.

04

Memory Dashboard
A local web dashboard at localhost:6173 gives you full visibility and control. See all memories on a timeline, search by keyword or file, pin critical memories that should always be injected, edit incorrect entries, and export your entire project memory as a portable .engram file.

05

Continuous Learning
Every session makes Engram smarter. The memory grows richer over time. The longer you use it, the more precisely Claude understands your project — your patterns, your preferences, your pitfalls, and your standards.

3.2 The Full Session Flow

→ Developer opens VS Code on their project

→ Engram extension activates automatically — detects project folder

→ Performs semantic search across memory store for current context

→	Retrieves top 15–20 most relevant memories
→	Constructs compact, intelligent context block (stays within token budget)
→	Injects context into Claude Code system prompt — invisible to developer
→	Developer types: "add appointment reminders" — no context given
→	Claude responds with full awareness of stack, schema, existing patterns
→	Session ends — file diffs are analyzed, new memories extracted and saved
→	Memory store grows richer. Next session starts smarter than this one.

4. Format Decision — Why VS Code Extension + Web Dashboard

The format of Engram was chosen deliberately. Every alternative was considered and rejected.

Format	Why Rejected	
Standalone Web App	Requires copy-pasting code into a browser. Breaks developer flow entirely.	✗ **Rejected**
Mobile App	Developers don't code on mobile. No access to local codebase or VS Code.	✗ **Rejected**
Pure CLI	Too cold for a hackathon demo. No visual memory surface. Hard to showcase.	✗ **Rejected**
Browser Extension	Cannot access local filesystem or run a vector DB. No codebase integration.	✗ **Rejected**
VS Code Extension + Web Dashboard	Lives inside the editor where developers work. Accesses filesystem. Can run local services. Zero friction.	✓ **Chosen**

> *The VS Code Extension captures sessions automatically with zero friction. The Web Dashboard makes memory visible and impressive during the hackathon demo. Together, they look and feel like a real, shippable product — not a hackathon hack.*

5. Technical Architecture & Stack

5.1 Full Technology Stack

Layer	Technology	Why This Choice	Version
Extension Core	**VS Code Extension API**	Native integration with VS Code — access to file watchers, workspace events, session hooks, and Claude Code MCP integration	VS Code 1.85+
Language	**TypeScript (strict)**	Type safety across the full extension codebase. Required for VS Code extension development. Catches memory-schema bugs at compile time.	TS 5.3+
Code Parser	**Tree-sitter**	Language-agnostic AST parser — handles JS, TS, Python, Go, Rust without custom parsers. Battle-tested in VS Code itself.	Latest
Vector DB	**ChromaDB (local)**	Runs 100% locally. No cloud dependency. Extremely fast semantic search. Simple Python API. Free and open-source.	v0.4+
Embeddings	**Claude API (Haiku)**	Fast and cheap embedding generation using claude-haiku. Consistent with the overall Claude API ecosystem used in this hackathon.	claude-haiku-4-5
AI Core	**Claude Sonnet 4**	Memory summarization, context compression, decision extraction from session diffs. Primary intelligence layer of Engram.	claude-sonnet-4-6
Memory Store	**SQLite + JSON**	Structured metadata alongside vector entries. Zero setup for end users. File-based — portable and backupable.	SQLite 3.x
Local Server	**Node.js + Express**	Lightweight bridge between VS Code extension (TypeScript) and ChromaDB (Python). Exposes REST API for dashboard.	Node 20 LTS
ChromaDB Bridge	**Python 3 + FastAPI**	ChromaDB's native SDK is Python. FastAPI exposes it as a local HTTP service consumed by the Node.js bridge.	Python 3.11+

Layer	Technology	Why This Choice	Version
Dashboard UI	**React + Vite + Tailwind**	Fast to build, fast to demo. Familiar stack. Runs on localhost:6173. Single-page app for memory visualization.	React 18
Extension Build	**esbuild**	Blazing fast bundler used officially for VS Code extensions. Produces single-file extension bundle.	Latest
Team Sync (stretch)	**Supabase**	Encrypted cloud sync for team memory sharing. E2E encrypted. Auth + realtime + storage in one.	Supabase v2

5.2 System Architecture Diagram (Described)

The Engram system consists of four interconnected services running locally on the developer's machine:

VS Code Extension	Node.js Bridge	ChromaDB Service	React Dashboard
TypeScript	Express REST API	Python + FastAPI	localhost:6173
Session hooks, file watchers, context injection, Tree-sitter parsing	*Memory write/read API, embedding generation via Claude, SQLite metadata*	*Vector store, semantic search, embedding storage, similarity ranking*	*Memory timeline, search, pin/edit/delete, export, import .engram files*

5.3 Memory Schema

Each memory entry stored in the system has the following structure:

Field	Type	Description
id	UUID	Unique identifier for the memory entry
type	Enum	bug_fix | decision | pattern | warning | preference | architecture
content	String	The actual memory text — what happened, what was decided, what was fixed
embedding	Float[]	Vector embedding of content — used for semantic similarity search
file_refs	String[]	Array of file paths this memory is linked to
line_refs	Int[]	Specific line numbers within referenced files (when relevant)
project_id	String	Project root path — scopes memories to the correct workspace
session_id	UUID	Claude Code session that generated this memory

Field	Type	Description
timestamp	DateTime	When this memory was created — used for recency weighting
pinned	Boolean	Pinned memories are always injected regardless of relevance score
tags	String[]	Optional developer-added tags for manual categorization
confidence	Float	0.0–1.0 score for how confident Engram is about this memory's signal quality

6. Core Feature Specifications

6.1 Codebase Indexer

Runs once on first activation. Produces a complete semantic map of the project.

Feature	Description	Priority
Directory Parser	Recursively scans project, respects .gitignore, maps full file tree with relationships	**P0 - Critical**
AST Code Analysis	Tree-sitter parses every JS/TS/Python file — extracts functions, components, classes, exports	**P0 - Critical**
DB Schema Detection	Reads migration files, Prisma/Drizzle schemas, Supabase types — maps all tables and relations	**P0 - Critical**
Route Mapping	Detects API routes, Expo Router pages, Next.js pages — builds navigation graph	**P0 - Critical**
Dependency Graph	Reads package.json / requirements.txt — identifies full dependency tree	**P1 - High**
Stack Fingerprinting	Auto-detects framework, DB, auth library, state manager, UI library from code patterns	**P1 - High**
Embedding Generation	Converts all extracted data to vector embeddings via Claude API (Haiku model)	**P0 - Critical**
Incremental Re-index	Watches for new files — re-indexes only changed files, not the full project	**P1 - High**
Multi-language Support	JS, TypeScript, Python supported in MVP. Go, Rust, Java in post-hackathon roadmap	**P1 - High**

6.2 Session Memory Recorder

Passively monitors Claude Code activity. Extracts meaningful memories without any developer input.

Feature	Description	Priority
File Diff Analyzer	After each session, computes git diff of changed files — sends to Claude for decision extraction	**P0 - Critical**
Decision Extractor	Claude Sonnet reads the diff + session context and identifies the key decision or fix made	**P0 - Critical**
Memory Tagging	Auto-tags each memory by type: bug_fix, decision, pattern, warning, or preference	**P0 - Critical**

Feature	Description	Priority
File Linker	Links each memory to the specific file(s) and line ranges it relates to	**P1 - High**
Deduplication	Semantic similarity check before storing — prevents near-duplicate memories from accumulating	**P1 - High**
Confidence Scoring	Scores each memory 0.0–1.0 on signal quality before storing — low-confidence entries flagged	**P1 - High**
Session Attribution	Records which session generated each memory — full audit trail	**P2 - Medium**
Manual Memory Entry	Developer can manually add memories via dashboard — for context not captured automatically	**P2 - Medium**

6.3 Intelligent Context Injector

The core engine. Runs at the start of every Claude Code session.

Feature	Description	Priority
Semantic Search	Finds memories most semantically similar to the current file and recent code changes	**P0 - Critical**
Recency Weighting	More recent memories score higher — older memories decay unless pinned	**P0 - Critical**
Token Budget Manager	Compresses context to stay within Claude's system prompt token limits intelligently	**P0 - Critical**
Pinned Memory Injection	Pinned memories always appear in context regardless of relevance score	**P0 - Critical**
Context Structuring	Formats injected context as structured markdown Claude can parse optimally	**P1 - High**
Project Fingerprint	Always injects a compact project summary: stack, key patterns, team conventions	**P0 - Critical**
Silent Operation	Zero UI friction — context injection is invisible, automatic, and instantaneous	**P0 - Critical**
Context Preview	Developer can preview what will be injected via dashboard before a session	**P2 - Medium**

6.4 Memory Dashboard (Web UI — localhost:6173)

Primary visualization and control surface. Also the main demo surface during hackathon judging.

Feature	Description	Priority
Memory Timeline	Chronological view of all memories — grouped by session, searchable, filterable	**P0 - Critical**
Memory Search	Full-text + semantic search across all memories by keyword, file, type, or date	**P0 - Critical**
Pin / Unpin	Pin critical memories to guarantee they're always injected into every session	**P0 - Critical**
Edit Memory	Correct or clarify any auto-generated memory entry directly in the dashboard	**P1 - High**
Delete Memory	Remove incorrect or outdated memories — keeps context fresh and accurate	**P1 - High**
Session Replay	View exactly which memories were injected in any past session	**P1 - High**
Export (.engram file)	Export full project memory as a portable .engram JSON file	**P1 - High**
Import (.engram file)	Import a teammate's .engram file — instant project onboarding for new devs	**P1 - High**
Project Stats	Memory count, coverage by file, most referenced components, session history chart	**P2 - Medium**
Memory Health Score	Dashboard score showing how comprehensive Engram's knowledge of the project is	**P2 - Medium**

6.5 Team Memory Sync (Stretch Goal — Post-Hackathon)

Feature	Description	Priority
Encrypted Cloud Sync	Supabase-backed sync of project memories — E2E encrypted, opt-in only	**P2 - Medium**
Shared + Personal Layers	Shared project memories + individual developer preferences — separate layers	**P2 - Medium**
New Dev Onboarding	New team member installs Engram, imports project — Claude knows everything instantly	**P2 - Medium**
Conflict Resolution	When two developers record conflicting decisions — flagged for manual resolution	**P3 - Low**
Access Controls	Project owner controls who can read and write team memories	**P3 - Low**

7. MVP Scope — Hackathon Submission

7.1 In Scope (Must Ship by April 28)

✓✓	VS Code extension activates automatically on project open
✓✓	Codebase indexer — JS and TypeScript support (covers DentaDesk demo project)
✓✓	Python support as secondary language
✓✓	Session memory recorder — file diff-based extraction via Claude Sonnet
✓✓	Context injector — automatic system prompt injection at session start
✓✓	Local web dashboard — memory timeline, search, pin, edit, delete
✓✓	Export / import .engram file
✓✓	Fully offline — all data local, zero cloud dependency for MVP
✓✓	Demo on DentaDesk project — 52 screens, real codebase, real memories
✓✓	Extension installable via VSIX file (no marketplace approval needed for demo)

7.2 Out of Scope for MVP

☐	Team memory cloud sync (Supabase) — post-hackathon v2.0
☐	Go, Rust, Java language support — post-hackathon
☐	VS Code Marketplace listing — requires review time
☐	GitHub branch-aware memory — memories scoped per git branch
☐	Multi-workspace support — single workspace only in MVP
☐	Auto-onboarding from git history — analyze commit log for past decisions
☐	JetBrains / Cursor IDE support — VS Code only for hackathon

8. Target Users & Personas

8.1 Primary User — Solo Developer

Attribute	Detail
Who	Solo developer building a SaaS, mobile app, or side project alone
Tool Use	Uses Claude Code daily — primary AI coding assistant
Pain	Constantly re-explains the same project to Claude every morning before any real work begins
Goal	Claude should feel like a senior dev who has been on the project since day one
Value Gain	Saves 10–20 minutes of context-setting per session — recovered immediately as productive coding time
Example	Abhi building DentaDesk — 52 screens, complex offline/sync architecture, weeks of accumulated decisions

8.2 Secondary User — Tech Lead

Attribute	Detail
Who	Leads a small engineering team of 2–8 developers all using Claude Code
Pain	Different developers give Claude different context — Claude gives inconsistent advice across the team
Goal	Consistent Claude behavior and architectural guidance for every team member on every session
Value Gain	Shared memory means Claude's suggestions are always aligned with team decisions and conventions
Example	A startup CTO who wants every engineer's Claude to know the system architecture and coding standards

8.3 Tertiary User — New Team Member

Attribute	Detail
Who	Developer who just joined a team or inherited a codebase
Pain	Traditional onboarding takes 2+ weeks of reading docs and asking senior devs questions
Goal	Be productive on day one — Claude should already know the project deeply
Value Gain	Import the team's .engram file — Claude immediately knows months of architectural history

Attribute	Detail
Example	A contractor brought in to extend DentaDesk — downloads .engram, Claude guides them instantly

8.4 Primary Demo Target — Hackathon Judge

Attribute	Detail
Who	Anthropic engineer or technical judge evaluating hackathon submissions
Context	Uses Claude Code personally every day — deeply familiar with the re-explanation pain
What Wins	Immediately recognizes the problem being solved as real and deeply personal to their own workflow
Demo Moment	Claude responds to a question about DentaDesk with full context — no context was manually given
Why They Remember It	They'll want to install it themselves during judging. That's the standard for a winning submission.

9. Success Metrics

9.1 Hackathon Success Criteria

Feature	Description	Priority
Context Injection Time	Full context injected into Claude session in under 2 seconds	**P0 - Critical**
Zero Manual Context	Demo session where developer types zero context — Claude responds correctly	**P0 - Critical**
Demo Reaction	At least one judge visibly reacts to Claude's contextual awareness during demo	**P0 - Critical**
Extension Install	Extension installs and activates on a fresh machine in under 5 minutes	**P0 - Critical**
Memory Accuracy	Injected memories are relevant to the current task > 90% of the time	**P1 - High**
Dashboard Load Time	Memory dashboard loads and renders full timeline in under 1 second	**P1 - High**

9.2 Post-Hackathon Product Metrics (6-Month Targets)

Feature	Description	Priority
VS Code Installs	10,000+ installs from VS Code Marketplace within 90 days of public launch	**P1 - High**
Daily Active Users	DAU/MAU ratio above 50% — developers use it every coding day	**P1 - High**
Session Time Saved	Average 15+ minutes saved per developer per day on context-setting	**P1 - High**
Memory Retention	Developers accumulate 100+ memories per project within first 2 weeks	**P2 - Medium**
NPS Score	Net Promoter Score above 50 within first 90 days of public release	**P2 - Medium**
GitHub Stars	1,000+ GitHub stars within 60 days of open-source release	**P2 - Medium**

10. Risks & Mitigations

Risk	Mitigation	Priority
ChromaDB Setup Complexity	Containerize ChromaDB in a Docker sidecar OR ship a pre-compiled binary — zero-config for end user	**P0 - Critical**
Context Token Overflow	Hard token limit on injected context (max 2000 tokens). Compression algorithm prioritizes pinned > recent > relevant	**P0 - Critical**
Claude Code API Hook	If Claude Code doesn't expose session hooks directly, use file system watchers + workspace change events as proxy	**P0 - Critical**
Privacy Concerns	All data is local by default. Explicit opt-in for any cloud features. README leads with privacy guarantee	**P1 - High**
Irrelevant Memory Injection	Confidence scoring + relevance threshold — memories below 0.6 similarity score are not injected	**P1 - High**
Index Performance on Large Repos	Incremental indexing — only re-index changed files. Skip node_modules, dist, build directories automatically	**P1 - High**
Memory Staleness	Timestamp-based decay scoring — old memories lose weight unless pinned or recently referenced	**P2 - Medium**
Hackathon Time Constraint	7-day hard deadline — MVP scoped tightly. Dashboard built last. Extension core ships first.	**P0 - Critical**

11. Why Engram Wins the Hackathon

Most hackathon entries will be applications built using Claude Code. Engram makes Claude Code itself better. That is a fundamentally different category of submission.

□	Judges feel this pain personally — every Anthropic engineer re-explains their codebase every morning
□	It's infrastructure, not an app — most entries build on top of Claude Code, Engram improves the foundation
⚡	The demo is undeniable — Claude responds with full project context without being given a single line of setup
□	Privacy-first — all memory stays local, no data leaves the machine, enterprise developers trust it immediately
□	Real product potential — VS Code Marketplace, 10k+ installs, potential YC application — judges see the runway
□	It solves a universal problem — not a niche use case, this affects every single Claude Code user every single day
□	The demo moment is instant — Claude knows DentaDesk's WatermelonDB schema, Razorpay integration, all 52 screens

12. The Winning Demo Script

The entire hackathon submission lives or dies on this single 90-second moment. Every line of code built in 7 days exists to make this moment happen.

> *Keep the demo under 90 seconds. The moment speaks for itself. Do not over-explain.*

Step 1	Open VS Code on the DentaDesk project — a fresh Claude Code session, no context given
Step 2	Show the empty session — say: "I'm not going to explain anything about this project to Claude"
Step 3	Type only: "add appointment reminder notifications to DentaDesk"
Step 4	Claude responds — mentioning WatermelonDB, Supabase Edge Functions, Expo Notifications, the existing notification service, the specific file structure — all without being told
Step 5	Open the Engram dashboard — show the memories that made this possible
Step 6	Say: "This is what Claude Code should have had from day one."
Step 7	Done. No further explanation needed.

13. Post-Hackathon Product Roadmap

13.1 v2.0 — Team Features

- Team memory cloud sync via Supabase (E2E encrypted, opt-in only)
- Shared project memory layer + individual developer preference layer
- New team member onboarding via .engram file import — zero ramp-up time
- GitHub integration — memories scoped to branches, PR-aware context

13.2 v3.0 — Intelligence Layer

- Git history auto-onboarding — analyze all commits and build initial memory from day one
- Multi-language support: Go, Rust, Java, Swift, Kotlin
- Memory conflict resolution — when two developers record contradictory decisions
- Memory analytics — what does Claude know most about? Gaps in coverage?
- Automatic memory decay and archiving — keeps context fresh without manual cleanup

13.3 v4.0 — Platform Expansion

- JetBrains IDE plugin (IntelliJ, WebStorm, PyCharm)
- Cursor IDE integration
- CLI version for terminal-based workflows
- Engram API — allow other tools to read/write project memories
- Paid Pro tier for team sync and advanced analytics

13.4 Monetization Strategy

Tier	Details
Free (OSS)	Single developer, local storage only, unlimited memories, full VS Code extension
Pro — $9/month	Team sync for up to 5 developers, encrypted cloud storage, priority support
Team — $29/month	Unlimited developers, admin controls, access controls, audit logs, SLA
Enterprise	On-premise deployment, SSO, custom integrations, dedicated support

14. Appendix

14.1 Competitive Landscape

Tool	What It Does	Why Engram Is Different
Cursor Memories	Stores some user preferences in Cursor IDE	Cursor-only, shallow, not semantic, not codebase-aware
Mem0	AI memory layer for chatbots and agents	Not VS Code native, not codebase-indexed, not Claude Code specific
Codebase AI Tools	RAG over codebase for Q&A (e.g. Greptile)	Search-only, no session memory, no context injection, no decision tracking
GitHub Copilot	In-editor autocomplete with repo context	Copilot reads files, not sessions. No cross-session decision memory.
Plain .cursorrules	Markdown file with project instructions for AI tools	Manual, static, doesn't grow, developer must write and maintain everything
Engram	Persistent semantic memory across all Claude Code sessions	**Automatic, semantic, grows over time, fully integrated, codebase-aware**

14.2 Glossary

Term	Definition
Engram	A memory trace stored in the brain — the name reflects Engram's purpose: persistent memory for Claude Code
Vector Embedding	A numerical representation of text that encodes semantic meaning — used for similarity search in ChromaDB
ChromaDB	An open-source vector database for storing and querying embedding-based memories locally
Tree-sitter	A language-agnostic code parsing library that produces Abstract Syntax Trees — used for codebase indexing
Context Injection	The act of automatically prepending relevant memories to Claude's system prompt at session start
Session	A single Claude Code conversation — Engram records memories from each session as it ends
Semantic Search	Finding memories based on meaning similarity rather than exact keyword match
Token Budget	The maximum number of tokens available in Claude's context window — Engram manages this carefully
.engram File	A portable JSON export of a project's full memory store — can be shared between developers

Term	Definition
Memory Decay	The process of reducing the relevance score of older memories over time unless they are pinned
MCP	Model Context Protocol — the standard Claude Code uses for tool and context integration
VSIX	VS Code extension package format — the file format used to install VS Code extensions